ECS 189G Spring 2026: Training and Probing a Tiny Transformer

TO PREPARE AND SUBMIT HOMEWORK

Follow these steps exactly, so your assignment can be properly graded. Failure to do so may result in a zero grade:

1. You must complete your implementations within the provided skeleton files: model.py and train.py.
2. You must compile your training loss curve (loss_curve.png), generated text samples (from inference.py), and your ablation study analysis into a single PDF file named Report.pdf.
3. Package your model.py, train.py, and Report.pdf into a single .zip archive.
4. Upload your .zip file to the course submission portal by the due date.

INSTRUCTIONS

In this assignment, you will build, train, and evaluate a minimal decoder-only Transformer from scratch using PyTorch. The goal is to build end-to-end familiarity with core Transformer components, the training pipeline, and empirical analysis practices.

We will train this model on the Tiny Shakespeare dataset (~1MB text corpus) to generate Shakespeare-like text. We will use character-level tokenization (vocabulary size ~65) to bypass complex BPE tokenizers and drastically reduce the embedding and linear layer parameters.

The default model configuration contains roughly 600K parameters and is designed to be CPU-friendly. You do not need a GPU (but could be better if you have) to complete the base requirements of this assignment; training the baseline model will take approximately 12 hours on a standard laptop CPU.

ENVIRONMENT SETUP

It is highly recommended to use a virtual environment (e.g., Conda or Python venv) to avoid dependency conflicts. We have provided a requirements.txt file for you.

MODEL IMPLEMENTATIONS [45 POINTS]

A skeleton file model.py containing empty definitions and TODO blocks has been provided. Since portions of this assignment depend on these specific architectures, none of the existing names or function signatures in this file should be modified.

In the Head class, implement the complete scaled dot-product attention mechanism:

- **[15 POINTS] Compute the scaled attention scores.**
- **[15 POINTS] Apply the causal mask.**
- **[15 POINTS] Compute the final output by multiplying the attention weights with the values.**

In the TransformerLanguageModel class, compute the Cross-Entropy Loss for the language modeling objective between the predicted logits and the target tokens.

You can verify your forward pass and loss computation by running the model directly. If your implementation is correct, it will print a success message and an initial loss value.

BASELINE TAINING & QUALITATIVE EVALUATION [45 POINTS]

Once the model architecture is complete, open train.py and locate the TODO block inside the training loop.

- **[15 POINTS] You must implement the standard PyTorch training steps:**
  1. Perform the forward pass.
  2. Zero the gradients.
  3. Perform the backward pass.
  4. Step the optimizer.
- **[15 POINTS]** With the model and training loop implemented, execute the training script. This script will train the model for 5000 iterations, plot the training and validation loss curves dynamically, and automatically save the best model weights to checkpoints/best_model.pt.
- **[15 POINTS]** Once training is complete, run the inference script to compare the generated text of an untrained model versus your newly trained model. Observe how the trained model learns the structure, character names, and basic vocabulary of Shakespearean plays. Due to the extremely small model capacity, limited dataset size, and short training duration, the trained model will likely not output perfectly fluent Shakespearean plays, and you will certainly encounter grammatical and spelling errors. However, compared to the completely random output of the untrained baseline model, the trained model's generated text should demonstrate discernible learned structures and formatting rules (e.g., character names, line breaks, and basic English word formations).

Include your loss_curve.png and a sample of your generated text in your Report.pdf.

ABLATION STUDIES & ANALYSIS [10 POINTS]

In this section, you will modify the hyperparameters at the top of train.py to conduct experiments and track the training dynamics. You must analyze how these changes impact both generation quality and computational overhead.

In your Report.pdf, provide accurate plots of comparison curves and discuss the following:

- **Effect of Context Length:** Change context_length (e.g., from 64 to 256). Observe and report the impact on the loss curve, training speed, and text generation quality.
- **Effect of Model Size:** Increase the model capacity parameters (n_embd from 128 to 256, n_head from 3 to 6, n_layer from 3 to 6, dropout from 0 to 0.1). Discuss the trade-offs between parameter count, convergence speed, and potential overfitting.